Validación Interactiva del Formulario de Productos con Programación Orientada a Objetos y Manejo de Eventos de Teclado y Mouse en JavaScript

[Nombre y apellido del estudiante]

Universidad Privada Domingo Savio

[Carrera]

[Asignatura]

[Nombre del docente]

6 de noviembre de 2026

Índice

Resumen 4

Validación Interactiva del Formulario de Productos 5

Objetivos 6

Marco Conceptual 6

Programación Orientada a Objetos en JavaScript 6

Características Modernas de JavaScript Investigadas 7

Modelo de Eventos del Navegador 8

Patrón Modelo–Vista–Controlador 9

Ventajas de Combinar POO y Eventos en la Validación 9

Validación en el Cliente y Seguridad 10

Desarrollo del Sistema 10

Estructura del Proyecto 10

Diagrama de Clases 11

Capa Modelo (modelo.js) 12

Capa Vista (vista.js) 16

Capa Controlador (controlador.js) 19

Resumen de Eventos Implementados 25

Flujo al Registrar un Producto 25

Base de Datos y Despliegue 26

Neon: PostgreSQL en la Nube 26

Modelo de Datos 28

Servidor y API REST 29

Validación en Tres Niveles 30

Diagrama de Secuencia: Guardar un Producto 31

Despliegue en Internet 32

Resultados y Pruebas 34

Funcionamiento del Sistema 34

Pruebas Funcionales 39

Uso de Inteligencia Artificial 41

Guía para la Defensa del Proyecto 42

Ejes Transversales y Objetivos de Desarrollo Sostenible 45

Conclusiones 45

Referencias 47

Resumen

El presente taller práctico desarrolla un sistema web de registro de productos cuyo formulario se valida de forma interactiva mediante Programación Orientada a Objetos (POO) y controladores de eventos de teclado y mouse en JavaScript puro. El código de la página se organizó con el patrón Modelo–Vista–Controlador (MVC) en tres archivos: `modelo.js` (datos y reglas), `vista.js` (elementos de pantalla) y `controlador.js` (eventos y coordinación). Se construyeron once clases que aplican los cuatro pilares de la POO: encapsulamiento mediante campos privados, abstracción mediante la clase abstracta `Regla`, herencia en dos jerarquías (reglas de validación y ventanas modales) y polimorfismo mediante la sobrescritura de métodos. La interacción se controla con los eventos `beforeinput`, `keydown`, `input`, `click`, `dblclick`, `mousedown`, `focusout`, `change` y `submit`, aplicando delegación de eventos y prevención de acciones predeterminadas. Los productos se almacenan en una base de datos PostgreSQL alojada en Neon, a la que la página accede mediante una API REST desarrollada con Node.js y Express; las reglas se validan en tres niveles (navegador, servidor y base de datos) y el sistema se publicó en internet con Render. La arquitectura se documenta con diagramas UML de clases, de secuencia y de despliegue, y con el modelo entidad–relación de la base de datos. Se ejecutaron 35 pruebas automatizadas en el navegador, con un 100 % de casos aprobados. Además, se documentan los prompts empleados con inteligencia artificial y una guía de preguntas para la defensa del proyecto.

Palabras clave: programación orientada a objetos, JavaScript, eventos del DOM, validación de formularios, patrón MVC, PostgreSQL, API REST, despliegue web.

Validación Interactiva del Formulario de Productos

Los formularios son el principal medio por el que un usuario introduce información en una aplicación web. Si los datos no se revisan en el momento en que se escriben, el sistema termina almacenando precios negativos, códigos repetidos o descripciones vacías, lo que afecta la calidad de la información y la toma de decisiones en un negocio. La validación interactiva resuelve este problema al revisar cada dato mientras el usuario escribe y al guiarlo con mensajes claros (Mozilla, s.f.-f).

El propósito de este taller es desarrollar módulos lógicos orientados a objetos y controladores de eventos de teclado y mouse para la validación interactiva de un formulario de productos. En lugar de escribir funciones sueltas, se modelaron los conceptos del problema como clases —`Producto`, `Regla`, `Campo`, `Inventario`, `Ventana`— que se relacionan entre sí. Esto permite que el código sea más ordenado, reutilizable y fácil de explicar, características que Flanagan (2020) y Haverbeke (2024) señalan como las principales ventajas de la POO en JavaScript moderno.

El documento se organiza en seis partes: el marco conceptual con la investigación de las características y ventajas de la POO y de los eventos; el desarrollo, donde se explica cada clase y sus métodos más importantes con el código fuente; la base de datos, el servidor y el despliegue del sistema en internet; las pruebas realizadas con sus capturas de pantalla; el uso de inteligencia artificial y los prompts redactados; y, finalmente, una guía de preguntas para la defensa del programa.

Objetivos

Objetivo general

Desarrollar un sistema web de registro de productos con validación interactiva del formulario, aplicando Programación Orientada a Objetos y controladores de eventos de teclado y mouse en JavaScript.

Objetivos específicos

- Investigar las características y ventajas de la POO y del modelo de eventos del navegador.
- Diseñar una arquitectura de clases con el patrón Modelo–Vista–Controlador.
- Implementar reglas de validación reutilizables mediante herencia y polimorfismo.
- Programar controladores de eventos de teclado y mouse que guíen al usuario.
- Almacenar los productos en una base de datos PostgreSQL mediante una API REST y publicar el sistema en internet.
- Verificar el funcionamiento del sistema mediante pruebas documentadas.

Marco Conceptual

Programación Orientada a Objetos en JavaScript

La POO es un paradigma que organiza el software en **objetos**: unidades que agrupan datos (atributos) y comportamiento (métodos). Una **clase** es la plantilla a partir de la cual se crean los objetos. Desde la especificación ECMAScript 2015, JavaScript incorpora la palabra reservada `class`, y versiones posteriores añadieron campos privados con el prefijo `#`, métodos estáticos y bloques de inicialización (Ecma International, 2024). Internamente las clases se apoyan en el sistema de prototipos del lenguaje, pero su sintaxis resulta más clara y cercana a otros lenguajes orientados a objetos (Mozilla, s.f.-a).

La Tabla 1 resume los cuatro pilares de la POO y la forma concreta en que se aplicaron en este proyecto.

Tabla 1

Pilares de la POO aplicados en el proyecto

Pilar	Definición	Aplicación en el proyecto
Encapsulamiento	Ocultar el estado interno de un objeto y exponer solo lo necesario.	Campos privados `#codigo`, `#precio`, `#listaProductos`; acceso solo mediante *getters*.
Abstracción	Representar un concepto general sin detallar su implementación.	La clase abstracta `Regla` define `cumple()` sin implementarlo; no puede instanciarse.
Herencia	Crear una clase nueva a partir de otra, reutilizando su código.	5 reglas heredan de `Regla`; `VentanaConfirmacion` y `VentanaMensaje` heredan de `Ventana`.
Polimorfismo	Un mismo mensaje produce comportamientos distintos según el objeto.	`Campo` llama `regla.cumple()` sin saber qué regla es; `VentanaConfirmacion` sobrescribe `cerrar()`.

Nota. Elaboración propia con base en Flanagan (2020) y Mozilla (s.f.-a).

Características Modernas de JavaScript Investigadas

Como parte de la investigación se estudiaron características recientes del lenguaje que fortalecen el diseño orientado a objetos. La Tabla 2 presenta cada una, su ventaja y dónde se utilizó.

Tabla 2

Características de JavaScript investigadas y su ventaja

Característica	Sintaxis	Ventaja	Uso en el proyecto
Campos privados	`#nombre`	Protección real del estado; el navegador impide el acceso externo.	`Producto`, `Inventario`, `Campo`, `Ventana`
Getters	`get valorTotal()`	Exponen datos de solo lectura o calculados.	`Producto.valorTotal`, `Campo.valor`
Herencia	`extends`, `super`	Reutiliza código y crea jerarquías.	Reglas y ventanas
Clase abstracta	`new.target`	Impide crear objetos de una clase base.	Constructor de `Regla`
Miembros estáticos	`static ICONOS`	Datos compartidos por la clase, no por cada objeto.	`VentanaMensaje.ICONOS`
Promesas y async/await	`new Promise`, `await`	Esperan una respuesta (del usuario o del servidor) sin bloquear la página.	`VentanaConfirmacion.preguntar()`, Inventario

Característica	Sintaxis	Ventaja	Uso en el proyecto
Encadenamiento opcional	`?.`	Evita errores si un objeto no existe.	Paso al siguiente campo con Enter
Peticiones HTTP	`fetch()`	Comunica la página con el servidor sin recargarla.	Inventario

Nota. Elaboración propia con base en Ecma International (2024) y Mozilla (s.f.-b, s.f.-g).

Modelo de Eventos del Navegador

Un **evento** es una señal que el navegador emite cuando ocurre algo: una tecla presionada, un clic, el envío de un formulario. El método `addEventListener(tipo, función)` registra una función, llamada *controlador* o *manejador*, que se ejecuta cuando ocurre ese evento (Mozilla, s.f.-c). El objeto `evento` que recibe la función contiene información como la tecla presionada (`evento.key`), el elemento que lo originó (`evento.target`) o si se mantenía presionada la tecla Ctrl (`evento.ctrlKey`).

Los eventos se **propagan** (burbujeo): un clic en un botón también llega a la fila, a la tabla y al documento. Esta propiedad permite la **delegación de eventos**, que consiste en registrar un solo controlador en un elemento contenedor y usar `evento.target` para saber qué hijo lo originó. La delegación reduce la cantidad de controladores y funciona aun con elementos creados después, como las filas que se agregan a la tabla (Mozilla, s.f.-d). Por otra parte, `evento.preventDefault()` cancela la acción predeterminada del navegador, por ejemplo, recargar la página al enviar un formulario o abrir “Guardar página” con Ctrl + S.

Tabla 3

Eventos de teclado investigados

Evento	Cuándo ocurre	¿Se usó?	Motivo
`keydown`	Al presionar una tecla.	Sí	Atajos: Enter, Esc, Ctrl + S, Alt + C.
`keyup`	Al soltar una tecla.	No	`input` cubre mejor los cambios de texto.
`beforeinput`	Antes de que el texto cambie.	Sí	Bloquear letras en precio y stock antes de que aparezcan.
`input`	Después de que el texto cambia.	Sí	Validar en vivo, contador, buscador y mayúsculas.

Evento	Cuándo ocurre	¿Se usó?	Motivo
`keypress`	Al producir un carácter.	No	Está obsoleto según la especificación.

Nota. Elaboración propia con base en Mozilla (s.f.-e) y W3C (2023).

Tabla 4

Eventos de mouse y de formulario investigados

Evento	Cuándo ocurre	Uso en el proyecto
`click`	Clic con el botón principal.	Nuevo producto, Guardar, Cancelar, Editar, Eliminar, "Sí, eliminar".
`dblclick`	Doble clic.	Editar un producto haciendo doble clic en su fila.
`mousedown`	Al presionar el botón del mouse.	Cerrar una ventana al hacer clic en el fondo oscuro.
`mouseover` / `mouseenter`	El puntero entra a un elemento (el primero burbujea, el segundo no).	Investigado; el resaltado de filas se resolvió con CSS `:hover`.
`focusout`	Un campo pierde el foco (burbujea, a diferencia de `blur`).	Validar el campo al salir de él.
`change`	Cambia la opción de una lista.	Validar la categoría.
`submit`	Se envía el formulario.	Validar todo y guardar.

Nota. Elaboración propia con base en Mozilla (s.f.-c, s.f.-d).

Patrón Modelo–Vista–Controlador

El patrón MVC separa una aplicación en tres responsabilidades: el **Modelo** contiene los datos y las reglas del negocio; la **Vista** muestra la información en pantalla; y el **Controlador** recibe las acciones del usuario y coordina a los otros dos (Gamma et al., 1994). Su ventaja es que cada parte puede modificarse sin afectar a las demás: por ejemplo, se puede cambiar el diseño de la tabla sin tocar las reglas de validación.

Ventajas de Combinar POO y Eventos en la Validación

- **Reutilización:** una misma clase `ReglaLongitud` valida el nombre y la descripción con distintos parámetros.
- **Extensibilidad:** agregar una regla nueva solo requiere crear otra subclase de `Regla`, sin modificar el código existente.

- **Mantenimiento:** cada clase tiene una única responsabilidad, por lo que los errores se ubican rápido.
- **Mejor experiencia de usuario:** los eventos permiten corregir errores en el momento, no después de enviar.
- **Accesibilidad:** el formulario puede usarse completo con el teclado, en línea con las pautas WCAG 2.2 (W3C, 2023).

Validación en el Cliente y Seguridad

La validación en el navegador mejora la experiencia del usuario, pero no reemplaza la validación en el servidor, porque cualquier persona puede desactivar o modificar el JavaScript de una página (OWASP Foundation, s.f.). Por ello, en este proyecto las mismas reglas se repiten en el servidor, con la función `validarProducto()`, y en la base de datos, con restricciones `CHECK`, como se explica en la sección Base de Datos y Despliegue. Además, los textos ingresados se insertan con `textContent` y no con `innerHTML`, lo que evita que un usuario inyecte código HTML o JavaScript en la tabla.

Desarrollo del Sistema

Estructura del Proyecto

El proyecto tiene dos partes: la página (HTML, CSS y JavaScript), que se ejecuta en el navegador, y un servidor en Node.js que guarda los datos en PostgreSQL. Para ejecutarlo en una computadora se usan los comandos `npm install` y `npm start` y se abre `http://localhost:3000`; además, el sistema está publicado en internet. La Tabla 5 describe cada archivo.

Tabla 5

Estructura de archivos del proyecto

Archivo	Capa	Contenido
`index.html`	Estructura	Página principal, tabla de productos y las tres ventanas: formulario, confirmación y mensaje.
`css/estilos.css`	Diseño	Estilos organizados en 11 secciones numeradas.
`js/modelo.js`	Modelo	`Producto`, `Regla` y sus 5 subclases, `Inventario`.
`js/vista.js`	Vista	`Campo`, `Ventana`, `VentanaConfirmacion`, `VentanaMensaje`, `TablaProductos`.
`js/controlador.js`	Controlador	`ControladorFormulario` y `Aplicacion`.
`servidor.js`	Servidor	API REST con Express: publica la página y lee y escribe los productos en PostgreSQL.
`base_datos.sql`	Base de datos	Script que crea las tablas categorias y productos con sus restricciones y datos de ejemplo.
`package.json`	Configuración	Dependencias (express y pg) y el comando de inicio.
`.env`	Configuración	Cadena de conexión DATABASE_URL. No se sube a GitHub (está en .gitignore).

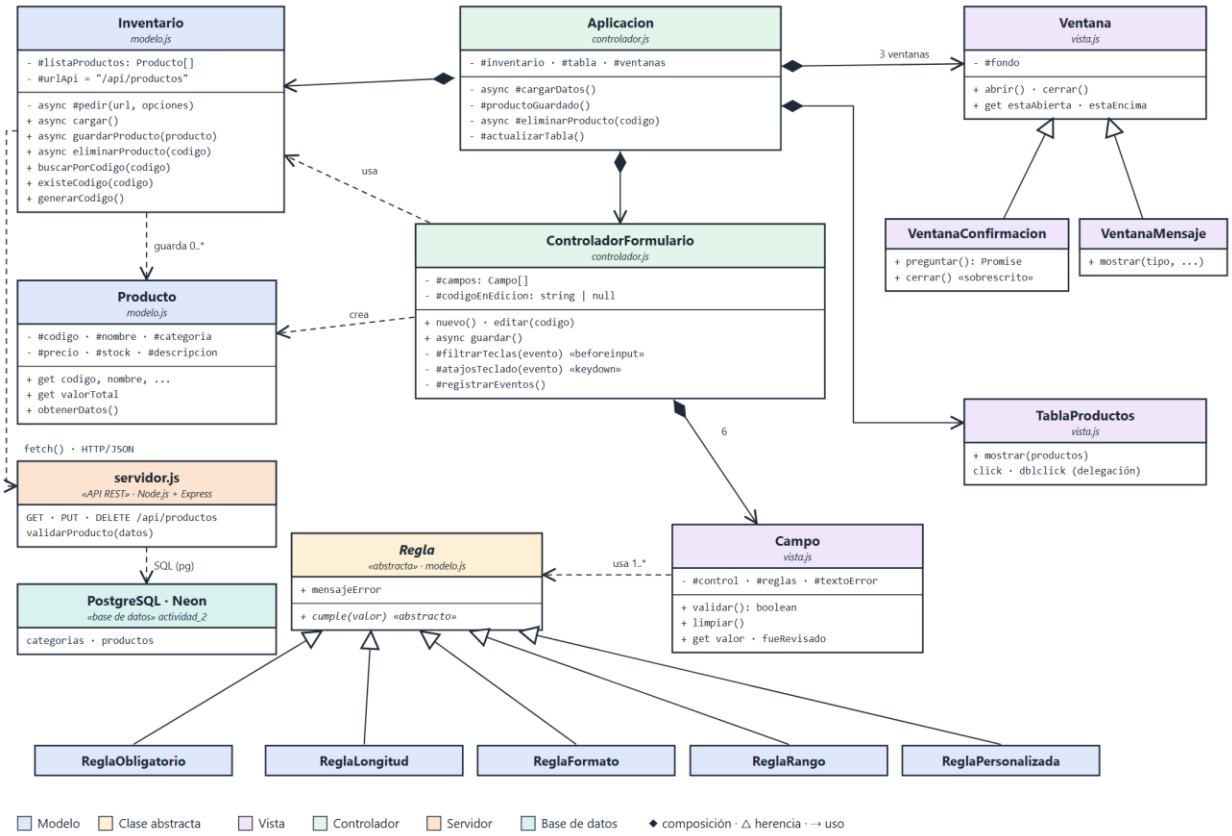
Nota. Los scripts de la página se cargan en orden: modelo → vista → controlador, porque cada capa usa las clases de la anterior.

Diagrama de Clases

La Figura 1 muestra las once clases y sus relaciones. Las flechas con triángulo indican herencia; las que tienen rombo, composición (una clase crea y contiene a otra); y las punteadas, uso o dependencia. Como referencia, abajo a la izquierda se incluyen el servidor (`servidor.js`) y la base de datos PostgreSQL en Neon, con los que `Inventario` se comunica mediante `fetch`.

Figura 1

Diagrama de clases UML del sistema



Nota. Elaboración propia. Colores: azul = Modelo, morado = Vista, verde = Controlador, amarillo = clase abstracta, naranja = servidor y turquesa = base de datos.

Capa Modelo (modelo.js)

Clase Producto

Representa un producto del inventario. Sus seis atributos son **privados** (`#`), por lo que nadie fuera de la clase puede cambiarlos directamente: solo se leen mediante *getters*. El constructor recibe un objeto y normaliza los datos: convierte el código a mayúsculas, elimina espacios sobrantes y transforma el precio y el stock a números. El *getter* `valorTotal` es una **propiedad calculada**: no se almacena, se obtiene multiplicando precio por stock cada vez que se consulta.

Figura 2

Clase Producto con campos privados y getters

```
class Producto {
  // Campos privados (#): solo se usan dentro de la clase (encapsulamiento).
  #codigo;
  #nombre;
  #categoria;
  #precio;
  #stock;
  #descripcion;

  constructor({ codigo, nombre, categoria, precio, stock, descripcion }) {
    this.#codigo = codigo.toUpperCase();
    this.#nombre = nombre.trim();
    this.#categoria = categoria;
    this.#precio = Number(precio);
    this.#stock = Number(stock);
    this.#descripcion = descripcion.trim();
  }

  // Getters: permiten leer los datos sin poder cambiarlos desde fuera.
  get codigo() { return this.#codigo; }
  get nombre() { return this.#nombre; }
  get categoria() { return this.#categoria; }
  get precio() { return this.#precio; }
  get stock() { return this.#stock; }
  get descripcion() { return this.#descripcion; }

  // Valor total del producto en el inventario.
  get valorTotal() {
    return this.#precio * this.#stock;
  }
```

Nota. Fragmento de `js/modelo.js`, líneas 6–35. El método `obtenerDatos()` (no mostrado) devuelve un objeto simple que se usa para enviar los datos al servidor y para llenar el formulario al editar.

Clase Abstracta Regla y sus Subclases

`Regla` es la base de todas las validaciones. Es **abstracta**: si alguien intenta crearla con `new Regla()`, el constructor detecta mediante `new.target` que no se usó una subclase y lanza un error. Define el atributo `mensajeError` y el método `cumple(valor)`, que cada subclase debe sobrescribir. Gracias a esto, la clase `Campo` puede revisar cualquier lista de reglas llamando a `cumple()` sin conocer qué tipo de regla es: eso es **polimorfismo**.

Figura 3

Clase abstracta Regla y las subclases ReglaObligatorio y ReglaLongitud

```
52 // Clase abstracta: no se puede crear directamente, solo sus hijas.
53 class Regla {
54   constructor(mensajeError) {
55     if (new.target === Regla) {
56       throw new Error('Regla es abstracta: use una subclase.');
57     }
58     this.mensajeError = mensajeError;
59   }
60
61   // Cada subclase escribe su propia versión (polimorfismo).
62   cumple(valor) {
63     throw new Error('El método cumple() debe implementarse en la subclase.');
64   }
65 }
66
67 // El campo no puede quedar vacío.
68 class ReglaObligatorio extends Regla {
69   cumple(valor) {
70     return valor !== '';
71   }
72 }
73
74 // El texto debe tener entre "minimo" y "maximo" caracteres.
75 class ReglaLongitud extends Regla {
76   constructor(minimo, maximo, mensajeError) {
77     super(mensajeError);
78     this.minimo = minimo;
79     this.maximo = maximo;
80   }
81
82   cumple(valor) {
83     return valor.length >= this.minimo && valor.length <= this.maximo;
84   }
85 }
```

Nota. Fragmento de `js/modelo.js`, líneas 52–85. Cada subclase llama a `super(mensajeError)` para reutilizar el constructor de la clase padre.

Las cinco subclases son: `ReglaObligatorio` (el campo no puede estar vacío), `ReglaLongitud` (cantidad mínima y máxima de caracteres), `ReglaFormato` (el texto debe cumplir una expresión regular), `ReglaRango` (el número debe estar entre un mínimo y un máximo) y `ReglaPersonalizada` (recibe una función con una condición propia, como verificar que el código no se repita).

Clase Inventario

Administra la lista de productos. Su atributo privado `#listaProductos` es un arreglo de objetos `Producto`. Los datos se guardan en la base de datos PostgreSQL: el método privado

`#pedir()` usa `fetch` para enviar peticiones HTTP a la API del servidor, y es el servidor quien ejecuta las consultas SQL (Mozilla, s.f.-g). Los métodos `cargar()`, `guardarProducto()` y `eliminarProducto()` son asíncronos (`async`/`await`): primero esperan la respuesta del servidor y, solo si fue correcta, actualizan la lista local. Si el servidor responde con un error, `#pedir()` lanza una excepción con el mensaje recibido, que la aplicación muestra en una ventana.

Figura 4

Comunicación de la clase Inventario con la API mediante fetch

```
132   // Envía una petición a la API y devuelve la respuesta en JSON.
133   async #pedir(url, opciones = {}) {
134     const respuesta = await fetch(url, {
135       headers: { 'Content-Type': 'application/json' },
136       ...opciones
137     });
138     const datos = await respuesta.json().catch(() => ({}));
139     if (!respuesta.ok) throw new Error(datos.error || 'No se pudo conectar con la base de datos.');
140     return datos;
141   }
142
143   // Lee los productos guardados en la base de datos.
144   async cargar() {
145     const filas = await this.#pedir(this.#urlApi);
146     this.#listaProductos = filas.map(datos => new Producto(datos));
147   }
```

Nota. Fragmento de `js/modelo.js`, líneas 132–147. `fetch` envía la petición HTTP y `await` espera la respuesta sin bloquear la página; `respuesta.ok` indica si el servidor respondió sin errores.

Tabla 6

Métodos públicos de la clase Inventario

Método	Qué hace
`cargar()`	Pide a la API los productos guardados y los convierte en objetos `Producto`.
`get productos`	Devuelve una copia de la lista (`[...lista]`) para que no se modifique desde fuera.
`buscarPorCodigo(codigo)`	Devuelve el producto con ese código o `undefined`.
`existeCodigo(codigo)`	Devuelve `true` si el código ya está registrado.
`guardarProducto(producto)`	Envía una petición `PUT` a la API; si el servidor la acepta, agrega el producto a la lista o lo reemplaza (edición).
`eliminarProducto(codigo)`	Envía una petición `DELETE` a la API y, si responde bien, quita el producto de la lista con `filter`.
`generarCodigo()`	Calcula el siguiente código libre: PRD-0001, PRD-0002, etc.

Nota. Elaboración propia.

Capa Vista (vista.js)

Clase Campo

Une un control del formulario (caja de texto, lista o área de texto) con su lista de reglas. Su método principal es `validar()`: recorre las reglas con `find` y se detiene en la primera que no se cumple. Luego **pinta el resultado**: agrega la clase CSS `incorrecto` (borde rojo) o `correcto` (borde verde) y escribe el mensaje de error debajo del campo. El *getter* `fueRevisado` indica si el campo ya se validó antes, lo que permite volver a validarlo en vivo mientras el usuario corrige.

Figura 5

Método validar() de la clase Campo

```
28   // Revisa las reglas en orden y muestra el primer error encontrado.
29   validar() {
30     const reglaFallida = this.#reglas.find(regla => !regla.cumple(this.valor));
31     this.#control.classList.toggle('incorrecto', Boolean(reglaFallida));
32     this.#control.classList.toggle('correcto', !reglaFallida);
33     this.#textoError.textContent = reglaFallida ? reglaFallida.mensajeError : '';
34     return !reglaFallida;
35   }
```

Nota. Fragmento de `js/vista.js`, líneas 28–35. `classList.toggle(clase, condicion)` agrega la clase si la condición es verdadera y la quita si es falsa.

Clase Ventana

Es la clase base de las tres ventanas modales. En su constructor registra tres controladores de eventos: **mouse** (`mousedown` en el fondo oscuro cierra la ventana), **mouse** (`click` en los botones con atributo `data-cerrar`, como la X y Cancelar) y **teclado** (`keydown` con la tecla Escape). El *getter* `estaEncima` comprueba que la ventana sea la última abierta, para que Esc cierre solo la ventana visible al frente y no todas a la vez.

Figura 6

Constructor de la clase Ventana con eventos de mouse y teclado

```
47  class Ventana {
48    #fondo;
49
50    constructor(fondo) {
51      this.#fondo = fondo;
52
53      // Mouse: clic en el fondo oscuro (fuera del cuadro) cierra la ventana.
54      fondo.addEventListener('mousedown', evento => {
55        if (evento.target === fondo) this.cerrar();
56      });
57
58      // Mouse: botones con el atributo data-cerrar (X y Cancelar).
59      fondo.querySelectorAll('[data-cerrar]').forEach(boton => {
60        boton.addEventListener('click', () => this.cerrar());
61      });
62
63      // Teclado: Esc cierra solo la ventana que está encima.
64      document.addEventListener('keydown', evento => {
65        if (evento.key === 'Escape' && this.estaEncima) this.cerrar();
66      });
67    }
```

Nota. Fragmento de `js/vista.js`, líneas 47–67. `evento.target === fondo` es verdadero solo si el clic fue en el fondo y no dentro del cuadro blanco.

Clase VentanaConfirmacion

Hereda de `Ventana` y agrega el método `preguntar(titulo, mensaje)`, que muestra "¿Eliminar producto?" y devuelve una **promesa** (`Promise`). La promesa queda pendiente hasta que el usuario elige: si presiona "Sí, eliminar" se resuelve con `true`; si presiona Cancelar, Esc o hace clic afuera, se resuelve con `false`. Para lograrlo, la clase **sobrescribe** el método `cerrar()` heredado: primero llama a `super.cerrar()` y luego responde `false`. Este es un ejemplo directo de polimorfismo.

Figura 7

Clase VentanaConfirmacion con promesa y método sobrescrito

```
 92  class VentanaConfirmacion extends Ventana {
 93    #responder = null;
 94
 95    constructor(fondo) {
 96      super(fondo);
 97      fondo.querySelector('[data-aceptar]').addEventListener('click', () =>
this.#terminar(true));
 98    }
 99
100    // Devuelve una promesa que se resuelve cuando el usuario elige.
101    preguntar(titulo, mensaje) {
102      this.fondo.querySelector('.ventana-titulo').textContent = titulo;
103      this.fondo.querySelector('.ventana-texto').textContent = mensaje;
104      this.abrir();
105      this.fondo.querySelector('[data-aceptar]').focus();
106      return new Promise(resolver => (this.#responder = resolver));
107    }
108
109    // Cerrar sin aceptar (Esc, Cancelar o clic afuera) cuenta como "No".
110    cerrar() {
111      super.cerrar();
112      this.#terminar(false);
113    }
114
115    #terminar(respuesta) {
116      if (!this.#responder) return;
117      const responder = this.#responder;
118      this.#responder = null;
119      super.cerrar();
120      responder(respuesta);
121    }
122  }
```

Nota. Fragmento de `js/vista.js`, líneas 92–122. El atributo privado `#responder` guarda la función que resuelve la promesa hasta que el usuario elige una opción.

Clase VentanaMensaje

También hereda de `Ventana`. Su método `mostrar(tipo, titulo, mensaje)` cambia el ícono según el tipo (`exito` ✓, `error` ✕, `aviso` !), muestra el texto y programa con `setTimeout` el cierre automático a los 2 segundos. Los íconos se guardan en el atributo **estático** `ICONOS`, que pertenece a la clase y no a cada objeto. Se usa para los mensajes “Producto agregado”, “Producto actualizado”, “Producto eliminado” y “Datos incorrectos”.

Clase TablaProductos

Dibuja las filas de la tabla con el método `mostrar(productos)`. Para los botones Editar y Eliminar usa **delegación de eventos**: en lugar de registrar un controlador por cada botón, registra

uno solo en el cuerpo de la tabla y, con `evento.target.closest()`, identifica qué botón y qué fila se presionó. Lo mismo hace con el **doble clic** para editar. Las celdas se llenan con `textContent`, que evita la inyección de código.

Figura 8

Delegación de eventos de mouse en la clase TablaProductos

```
148   constructor(cuerpoTabla, acciones) {
149     this.#cuerpoTabla = cuerpoTabla;
150     this.#acciones = acciones;
151
152     // Mouse: un solo evento para todos los botones (delegación de eventos).
153     cuerpoTabla.addEventListener('click', evento => {
154       const boton = evento.target.closest('button[data-accion]');
155       if (!boton) return;
156       const codigo = boton.closest('tr').dataset.codigo;
157       this.#acciones[boton.dataset.accion](codigo);
158     });
159
160     // Mouse: doble clic sobre una fila para editarla.
161     cuerpoTabla.addEventListener('dblclick', evento => {
162       const fila = evento.target.closest('tr[data-codigo]');
163       if (fila) this.#acciones.editar(fila.dataset.codigo);
164     });
165   }
```

Nota. Fragmento de `js/vista.js`, líneas 148–165. `this.#acciones[boton.dataset.accion](codigo)` llama a `editar` o `eliminar` según el atributo `data-accion` del botón.

Capa Controlador (controlador.js)

Clase ControladorFormulario

Es el núcleo de la validación interactiva. En el método privado `#crearCampos()` se crea un objeto `Campo` por cada control del formulario y se le asignan sus reglas. Por ejemplo, el código tiene tres reglas: obligatorio, formato `PRD-0000` y no repetido. Esta última usa una `ReglaPersonalizada` con una función que consulta al inventario y permite el mismo código cuando se está editando ese producto.

Figura 9

Asignación de reglas a los campos en ControladorFormulario

```
29    #crearCampos() {
30      const controles = this.#formulario.elements;
31
32      this.#campos = [
33        new Campo(controles.codigo, [
34          new ReglaObligatorio('El código es obligatorio.'),
35          new ReglaFormato(/^PRD-\d{4}$/, 'Use el formato PRD-0001.'),
36          new ReglaPersonalizada(
37            codigo => codigo === this.#codigoEnEdicion ||
!this.#inventario.existeCodigo(codigo),
38            'Ya existe un producto con este código.'
39          )
40        ]),
41        new Campo(controles.categoria, [
42          new ReglaObligatorio('Seleccione una categoría.')
43        ]),
44        new Campo(controles.nombre, [
45          new ReglaObligatorio('El nombre es obligatorio.'),
46          new ReglaLongitud(3, 60, 'Debe tener entre 3 y 60 caracteres.')
47        ]),
```

Nota. Fragmento de `js/controlador.js`, líneas 29–47. El resto de campos (precio, stock y descripción) siguen la misma estructura.

Tabla 7

Reglas de validación por campo

Campo	Reglas aplicadas	Mensaje de error de ejemplo
Código	Obligatorio · Formato `^PRD-\d{4}$` · Personalizada (no repetido)	Use el formato PRD-0001.
Categoría	Obligatorio	Seleccione una categoría.
Nombre	Obligatorio · Longitud 3–60	Debe tener entre 3 y 60 caracteres.
Precio	Obligatorio · Formato (máx. 2 decimales) · Rango 0.10–100 000	Máximo 2 decimales (ej. 25.50).
Stock	Obligatorio · Formato (entero) · Rango 0–10 000	Debe estar entre 0 y 10 000.
Descripción	Obligatorio · Longitud 10–200	Debe tener entre 10 y 200 caracteres.

Nota. Las reglas se evalúan en orden y se muestra solo el primer error de cada campo.

El método `#registrarEventos()` conecta los controladores de eventos del formulario. Todos se registran **una sola vez sobre el formulario** y no sobre cada campo, aprovechando la propagación de eventos: `evento.target` indica qué campo los originó.

Figura 10

Registro de eventos del formulario

```
65   #registrarEventos() {
66     // Teclado: bloquea caracteres no permitidos antes de que se escriban.
67     this.#formulario.addEventListener('beforeinput', evento => this.#filtrarTeclas(evento));
68
69     // Teclado: Enter avanza al siguiente campo y Ctrl + S guarda.
70     this.#formulario.addEventListener('keydown', evento => this.#atajosTeclado(evento));
71
72     // Al escribir: código en mayúsculas, contador y validación en vivo.
73     this.#formulario.addEventListener('input', evento => {
74       const campo = this.#buscarCampo(evento.target);
75       if (!campo) return;
76       if (campo.nombre === 'codigo') campo.valor = campo.control.value.toUpperCase();
77       if (campo.nombre === 'descripcion') this.#actualizarContador();
78       if (campo.fueRevisado) campo.validar();
79     });
80
81     // Al salir de un campo que tiene datos, se valida.
82     this.#formulario.addEventListener('focusout', evento => {
83       const campo = this.#buscarCampo(evento.target);
84       if (campo && campo.valor !== '') campo.validar();
85     });
86
87     // La lista de categorías se valida apenas cambia.
88     this.#formulario.addEventListener('change', evento => {
89       if (evento.target.name === 'categoria') this.#buscarCampo(evento.target).validar();
90     });
91
92     // Mouse (clic en Guardar) o Enter en el botón.
93     this.#formulario.addEventListener('submit', evento => {
94       evento.preventDefault(); // evita que la página se recargue
95       this.guardar();
96     });
97   }
```

Nota. Fragmento de `js/controlador.js`, líneas 65–97. Se usa `focusout` en lugar de `blur` porque `focusout` sí burbujea hasta el formulario.

Los dos controladores de teclado más importantes se muestran en la Figura 11. `#filtrarTeclas()` se ejecuta en el evento `beforeinput`, es decir, **antes** de que el carácter aparezca: si el campo tiene `data-solo="entero"` o `data-solo="decimal"` y lo escrito no es un dígito (o un único punto en el precio), cancela la escritura con `preventDefault()`. `#atajosTeclado()` atiende Ctrl + S para guardar y Enter para pasar al siguiente campo.

Figura 11

Controladores de teclado: filtro de caracteres y atajos

```
  // Solo deja escribir números (y un punto decimal en el precio).
  #filtrarTeclas(evento) {
    const tipoPermitido = evento.target.dataset.solo; // "entero" o "decimal"
    if (!tipoPermitido || !evento.data) return;

    const esDigito = /^\d+$/.test(evento.data);
    const esPuntoValido = evento.data === '.' && tipoPermitido === 'decimal'
      && !evento.target.value.includes('.');

    if (!esDigito && !esPuntoValido) evento.preventDefault();
  }

  #atajosTeclado(evento) {
    // Ctrl + S: guarda (y evita "Guardar página" del navegador).
    if (evento.ctrlKey && evento.key.toLowerCase() === 's') {
      evento.preventDefault();
      this.guardar();
      return;
    }

    // Enter: pasa al siguiente campo (en la descripción hace salto de línea).
    if (evento.key === 'Enter' && evento.target.matches('input, select')) {
      evento.preventDefault();
      const posicion = this.#campos.findIndex(campo => campo.control === evento.target);
      this.#campos[posicion + 1]?.control.focus();
    }
  }
```

Nota. Fragmento de `js/controlador.js`, líneas 99–125. `evento.data` contiene el carácter que se intenta escribir.

Finalmente, el método `guardar()`, que ahora es asíncrono, valida todos los campos. Se usa `map` y no `every` a propósito: `every` se detendría en el primer error, mientras que `map` ejecuta todas las validaciones para que el usuario vea todos los errores a la vez. Si hay errores, enfoca el primer campo incorrecto y muestra el mensaje “Datos incorrectos”; si todo es correcto, crea un objeto `Producto`, espera con `await` a que el inventario lo guarde en la base de datos y recién entonces cierra la ventana. El atributo `#guardando` evita que el producto se envíe dos veces si el usuario presiona Guardar repetidamente.

Figura 12

Método guardar() de ControladorFormulario

```
150   // Valida todo; si está correcto, guarda el producto y cierra la ventana.
151   async guardar() {
152     if (this.#guardando) return; // evita guardar dos veces seguidas
153
154     // map (y no every) para que se muestren todos los errores a la vez.
155     const resultados = this.#campos.map(campo => campo.validar());
156
157     if (resultados.includes(false)) {
158       this.#campos[resultados.indexOf(false)].control.focus();
159       this.#alFallar();
160       return;
161     }
162
163     const datos = {};
164     this.#campos.forEach(campo => (datos[campo.nombre] = campo.valor));
165     const producto = new Producto(datos);
166
167     // Se guarda en la base de datos; si falla, la ventana queda abierta.
168     this.#guardando = true;
169     try {
170       await this.#inventario.guardarProducto(producto);
171     } catch (error) {
172       this.#alFallar('Error al guardar', error.message);
173       return;
174     } finally {
175       this.#guardando = false;
176     }
177
178     this.#ventana.cerrar();
179     this.#alGuardar(producto, this.#codigoEnEdicion !== null);
180   }
```

Nota. Fragmento de `js/controlador.js`, líneas 150–180. Si el servidor rechaza los datos, el bloque `catch` muestra el error y la ventana sigue abierta. `#alGuardar` y `#alFallar` son funciones que recibe el controlador al crearse; así no depende directamente de las ventanas de mensaje.

Tabla 8

Métodos de la clase ControladorFormulario

Método	Tipo	Qué hace
`nuevo()`	Público	Limpia el formulario, genera el código y abre la ventana “Nuevo producto”.
`editar(codigo)`	Público	Llena el formulario con el producto, bloquea el código y abre “Editar producto”.
`guardar()`	Público	Valida todo y guarda, o muestra los errores.
`#crearCampos()`	Privado	Crea los 6 objetos `Campo` con sus reglas.
`#registrarEventos()`	Privado	Registra `beforeinput`, `keydown`, `input`, `focusout`, `change` y `submit`.
`#filtrarTeclas(evento)`	Privado	Bloquea caracteres no permitidos.
`#atajosTeclado(evento)`	Privado	Ctrl + S guarda; Enter pasa al siguiente campo.
`#actualizarContador()`	Privado	Muestra los caracteres usados en la descripción (ej. 48/200).

Nota. Los métodos privados (`#`) solo pueden llamarse desde dentro de la clase.

Clase Aplicacion

Es el punto de entrada. Crea todos los objetos (inventario, ventanas, tabla y controlador del formulario) y los conecta; a esto se le llama composición. Al iniciar, su método asíncrono `#cargarDatos()` pide los productos a la base de datos; si no hay conexión, muestra el mensaje «Sin conexión». También registra el clic del botón "+ Nuevo producto", el evento `input` del buscador, que filtra la tabla mientras se escribe, y el atajo global Alt + C. Su método `#eliminarProducto()` es asíncrono: con `await` espera primero la respuesta de la ventana de confirmación y luego la del servidor.

Figura 13

Eliminación con confirmación asíncrona en la clase Aplicacion

```
270    // Pide confirmación antes de borrar.
271    async #eliminarProducto(codigo) {
272      const producto = this.#inventario.buscarPorCodigo(codigo);
273      const confirmado = await this.#ventanaConfirmacion.preguntar(
274        '¿Eliminar producto?',
275        `Se eliminará "${producto.nombre}" (${codigo}). Esta acción no se puede deshacer.`
276      );
277      if (!confirmado) return;
278
279      try {
280        await this.#inventario.eliminarProducto(codigo);
281      } catch (error) {
282        this.#ventanaMensaje.mostrar('error', 'Error al eliminar', error.message, 3000);
283        return;
284      }
285      this.#actualizarTabla();
286      this.#ventanaMensaje.mostrar('exito', 'Producto eliminado', `${producto.nombre} se eliminó
correctamente.`);
```

Nota. Fragmento de `js/controlador.js`, líneas 270–286. Si el usuario cancela, `confirmado` es `false` y la función termina sin eliminar; si el servidor no puede borrar, se muestra «Error al eliminar».

La última línea del archivo, `document.addEventListener('DOMContentLoaded', () => new Aplicacion())`, crea la aplicación cuando el navegador termina de construir la página.

Resumen de Eventos Implementados

Tabla 9

Controladores de eventos de teclado y mouse implementados

Tipo	Evento	Elemento	Clase y método	Acción
Teclado	`beforeinput`	Formulario	ControladorFormulario `#filtrarTeclas`	Bloquea letras en precio y stock
Teclado	`keydown`	Formulario	ControladorFormulario `#atajosTeclado`	Ctrl + S guarda · Enter avanza
Teclado	`keydown`	Documento	Ventana (constructor)	Esc cierra la ventana de arriba
Teclado	`keydown`	Documento	Aplicacion `#registrarEventos`	Alt + C abre "Nuevo producto"
Teclado	`input`	Formulario	ControladorFormulario	Mayúsculas, contador, validación en vivo
Teclado	`input`	Buscador	Aplicacion `#actualizarTabla`	Filtra la tabla
Mouse	`click`	Botón Nuevo	Aplicacion	Abre el formulario
Mouse	`click`	Cuerpo de tabla	TablaProductos (delegación)	Editar o Eliminar
Mouse	`click`	Botones X, Cancelar, Sí	Ventana / VentanaConfirmacion	Cierra o confirma
Mouse	`dblclick`	Cuerpo de tabla	TablaProductos (delegación)	Edita la fila
Mouse	`mousedown`	Fondo oscuro	Ventana	Clic afuera cierra la ventana
Formulario	`focusout` · `change` · `submit`	Formulario	ControladorFormulario	Valida al salir, al elegir y al guardar

Nota. Elaboración propia.

Flujo al Registrar un Producto

Para comprender cómo colaboran las clases, se describe paso a paso lo que ocurre cuando el usuario registra un producto:

1. El usuario hace clic en "+ Nuevo producto" (o presiona Alt + C). `Aplicacion` llama a `ControladorFormulario.nuevo()`.
2. `nuevo()` limpia los campos, pide a `Inventario.generarCodigo()` el siguiente código y abre la `Ventana` del formulario.
3. Mientras el usuario escribe, `beforeinput` bloquea los caracteres inválidos e `input` revalida los campos ya revisados.

4. Al salir de cada campo, `focusout` ejecuta `Campo.validar()`, que recorre sus objetos `Regla` llamando a `cumple()`.
5. Al presionar Guardar (o Ctrl + S), `guardar()` valida los seis campos. Si alguno falla, `VentanaMensaje` muestra “Datos incorrectos”.
6. Si todo es correcto, se crea un `Producto` e `Inventario.guardarProducto()` lo envía al servidor, que lo valida otra vez y lo guarda en la base de datos PostgreSQL.
7. La ventana se cierra, `TablaProductos.mostrar()` redibuja la tabla y `VentanaMensaje` muestra “Producto agregado” durante 2 segundos.

Base de Datos y Despliegue

En la primera versión, los productos se guardaban en el `localStorage` del navegador, por lo que cada computadora tenía sus propios datos y se perdían al borrar el historial. Para que la información sea compartida y permanente, el sistema se amplió con una base de datos PostgreSQL alojada en la nube (Neon) y un servidor en Node.js que actúa como intermediario. La página nunca se conecta directamente a la base de datos: envía peticiones HTTP al servidor y este ejecuta las consultas SQL. Así, la contraseña de la base de datos no llega al navegador.

Neon: PostgreSQL en la Nube

Neon es un servicio de PostgreSQL *serverless* en la nube (Neon, s.f.). A diferencia de un servidor de base de datos tradicional, separa el almacenamiento del cómputo: los datos se guardan de forma permanente, mientras que el cómputo, es decir, la máquina que ejecuta las consultas, se enciende cuando llega una consulta y se suspende cuando no hay actividad. Por eso su plan gratuito es suficiente para un proyecto académico. Además, permite crear ramas (*branches*) de la base de datos, de forma parecida a las ramas de Git, y ofrece un editor SQL en el navegador.

Para este proyecto se creó en Neon el proyecto «Actividad 2» y, dentro de él, la base de datos `actividad_2`, cuyo propietario es el rol `actividad_2_owner`. Luego se ejecutó el script `base_datos.sql` en el editor SQL de Neon para crear las tablas y cargar los datos iniciales. Por último, la cadena de conexión que entrega Neon se copió al archivo `.env`, para trabajar en la computadora, y a una variable de entorno de Render, para el sistema publicado. La Tabla 10 resume la configuración.

Tabla 10

Configuración del proyecto en Neon

Elemento	Valor
Proyecto	Actividad 2
Proveedor y región	AWS us-east-2 (Ohio, Estados Unidos)
Motor	PostgreSQL 18.6
Base de datos	`actividad_2`
Rol propietario	`actividad_2_owner`
Cómputo	0.25 CU (unidades de cómputo); se suspende sin actividad y se reactiva con la siguiente consulta.
Almacenamiento	Hasta 1 GB por rama en el plan gratuito; la base ocupa 7.7 MB.
Conexión	Servidor con sufijo `-pooler` (PgBouncer) y cifrado SSL (`sslmode=verify-full`).
Contenido	6 categorías y 3 productos de ejemplo.

Nota. Elaboración propia con datos de la consola de Neon, consultados el 6 de noviembre de 2026.

La cadena de conexión reúne en una sola línea todo lo necesario para conectarse; su estructura se explica en la Figura 14. La contraseña se reemplazó por asteriscos porque nunca debe publicarse. El sufijo `-pooler` indica que las conexiones pasan por PgBouncer, que las reutiliza para atender más peticiones, y `sslmode=verify-full` obliga a cifrar la comunicación y a verificar el certificado del servidor.

Figura 14

Partes de la cadena de conexión DATABASE_URL

```
postgresql://                → protocolo (PostgreSQL)
actividad_2_owner            → usuario (rol propietario)
:********                    → contraseña (secreta)
@ep-green-bread-be4slw07-pooler.c-9.us-east-2.aws.neon.tech
                             → servidor de Neon con pooler
/actividad_2                 → nombre de la base de datos
?sslmode=verify-full         → conexión cifrada con SSL
```

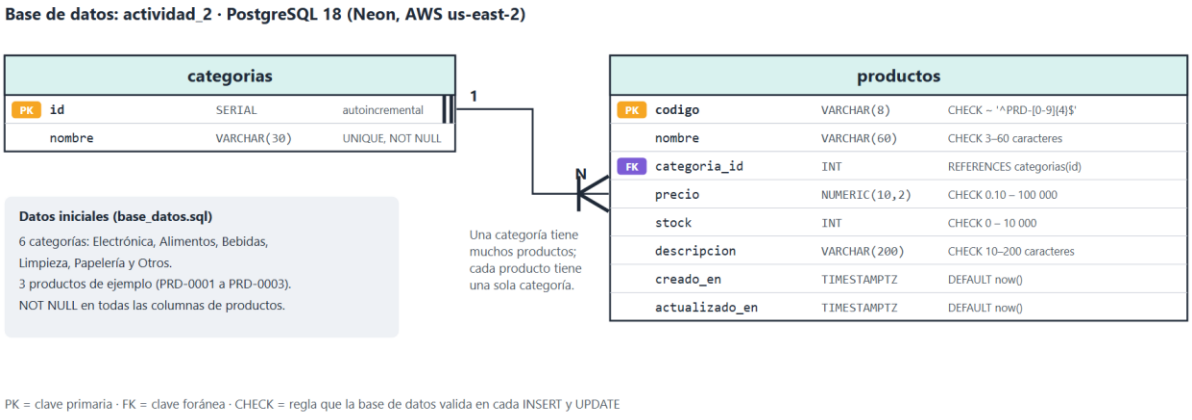
Nota. Elaboración propia. En el archivo `.env` la cadena se escribe en una sola línea: `DATABASE_URL=postgresql://…`

Modelo de Datos

La base de datos tiene dos tablas relacionadas, como muestra el modelo entidad–relación de la Figura 15: `categorias`, que guarda las seis categorías del formulario, y `productos`, que guarda cada producto. La relación es de uno a muchos: una categoría puede tener muchos productos, pero cada producto pertenece a una sola categoría a través de la clave foránea `categoria_id`. Guardar las categorías en su propia tabla evita repetir el texto en cada producto y permite agregar categorías nuevas sin cambiar el código.

Figura 15

Modelo entidad–relación de la base de datos actividad_2



Nota. Elaboración propia. PK = clave primaria; FK = clave foránea; 1 a N = relación de uno a muchos.

La Tabla 11 detalla las columnas de `productos` y la Figura 16 muestra el código SQL que crea la tabla. Las restricciones `CHECK` repiten las reglas del formulario, de modo que la base de

datos rechaza cualquier dato inválido aunque llegue sin pasar por la página (PostgreSQL Global Development Group, s.f.).

Tabla 11

Columnas de la tabla productos

Columna	Tipo	Restricción
codigo	VARCHAR(8)	Clave primaria; formato PRD-0000.
nombre	VARCHAR(60)	Obligatorio; entre 3 y 60 caracteres.
categoria_id	INT	Clave foránea a categorias(id).
precio	NUMERIC(10,2)	Entre 0.10 y 100 000, con 2 decimales.
stock	INT	Entero entre 0 y 10 000.
descripcion	VARCHAR(200)	Obligatorio; entre 10 y 200 caracteres.
creado_en	TIMESTAMPTZ	Fecha de registro automática (now()).
actualizado_en	TIMESTAMPTZ	Fecha de la última edición.

Nota. Elaboración propia con base en el archivo `base_datos.sql`.

Figura 16

Creación de la tabla productos con sus restricciones

```
17 CREATE TABLE IF NOT EXISTS productos (
18   codigo         VARCHAR(8)    PRIMARY KEY CHECK (codigo ~ '^PRD-[0-9]{4}$'),
19   nombre         VARCHAR(60)   NOT NULL CHECK (char_length(nombre) BETWEEN 3 AND 60),
20   categoria_id   INT           NOT NULL REFERENCES categorias(id),
21   precio         NUMERIC(10,2) NOT NULL CHECK (precio BETWEEN 0.10 AND 100000),
22   stock          INT           NOT NULL CHECK (stock BETWEEN 0 AND 10000),
23   descripcion    VARCHAR(200)  NOT NULL CHECK (char_length(descripcion) BETWEEN 10 AND 200),
24   creado_en      TIMESTAMPTZ   NOT NULL DEFAULT now(),
25   actualizado_en TIMESTAMPTZ   NOT NULL DEFAULT now()
26 );
```

Nota. Fragmento de `base_datos.sql`, líneas 17–26. El operador `~` compara el código con una expresión regular y `BETWEEN` verifica que el valor esté dentro del rango.

Servidor y API REST

El archivo `servidor.js` usa Express (OpenJS Foundation, s.f.) para publicar la página y una API REST, y la librería `pg` para conectarse a PostgreSQL. La cadena de conexión se lee de la variable de entorno `DATABASE_URL`. Las consultas usan parámetros (`$1`, `$2`…) en lugar de unir textos, lo que evita la inyección SQL. La Tabla 12 resume las rutas.

Tabla 12
Rutas de la API REST

Método y ruta	Acción	Respuesta
`GET /api/categorias`	Lista las categorías.	Arreglo con los nombres.
`GET /api/productos`	Lista los productos con su categoría.	Arreglo de productos.
`PUT /api/productos/:codigo`	Crea el producto o lo actualiza si ya existe.	201 creado, 200 actualizado o 400 con el error.
`DELETE /api/productos/:codigo`	Elimina el producto.	200 eliminado o 404 si no existe.

Nota. Elaboración propia con base en `servidor.js`.

La Figura 17 muestra la ruta `PUT`. Primero valida los datos con `validarProducto()` y, si hay errores, responde con el código 400. Luego ejecuta una sola consulta que inserta el producto o, si el código ya existe, lo actualiza (`ON CONFLICT DO UPDATE`). La expresión `xmax = 0` indica si la fila es nueva, para responder 201 (creado) o 200 (actualizado).

Figura 17
Ruta PUT del servidor: validación y guardado en PostgreSQL

```
58  // PUT /api/productos/:codigo → crea o actualiza (upsert)
59  app.put('/api/productos/:codigo', async (req, res, next) => {
60    const datos = { ...req.body, codigo: req.params.codigo.toUpperCase() };
61    const errores = validarProducto(datos);
62    if (errores.length) return res.status(400).json({ error: errores.join(' ') });
63
64    try {
65      const { rows, rowCount } = await baseDatos.query(
66        `INSERT INTO productos (codigo, nombre, categoria_id, precio, stock, descripcion)
67         SELECT $1, $2, c.id, $4, $5, $6 FROM categorias c WHERE c.nombre = $3
68         ON CONFLICT (codigo) DO UPDATE SET
69           nombre = EXCLUDED.nombre, categoria_id = EXCLUDED.categoria_id,
70           precio = EXCLUDED.precio, stock = EXCLUDED.stock,
71           descripcion = EXCLUDED.descripcion, actualizado_en = now()
72         RETURNING (xmax = 0) AS es_nuevo`,
73        [datos.codigo, datos.nombre.trim(), datos.categoria, Number(datos.precio), Number(datos.stock),
datos.descripcion.trim()]
74      );
75      if (rowCount === 0) return res.status(400).json({ error: 'La categoría no existe.' });
76      res.status(rows[0].es_nuevo ? 201 : 200).json({ ok: true });
77    } catch (error) { next(error); }
78  });
```

Nota. Fragmento de `servidor.js`, líneas 58–78. `$1`…`$6` son parámetros: `pg` envía los valores por separado de la consulta, lo que evita la inyección SQL.

Validación en Tres Niveles

Con el servidor y la base de datos, las reglas del formulario se aplican en tres niveles. En el navegador, las clases `Regla` validan mientras el usuario escribe y ofrecen la mejor experiencia.

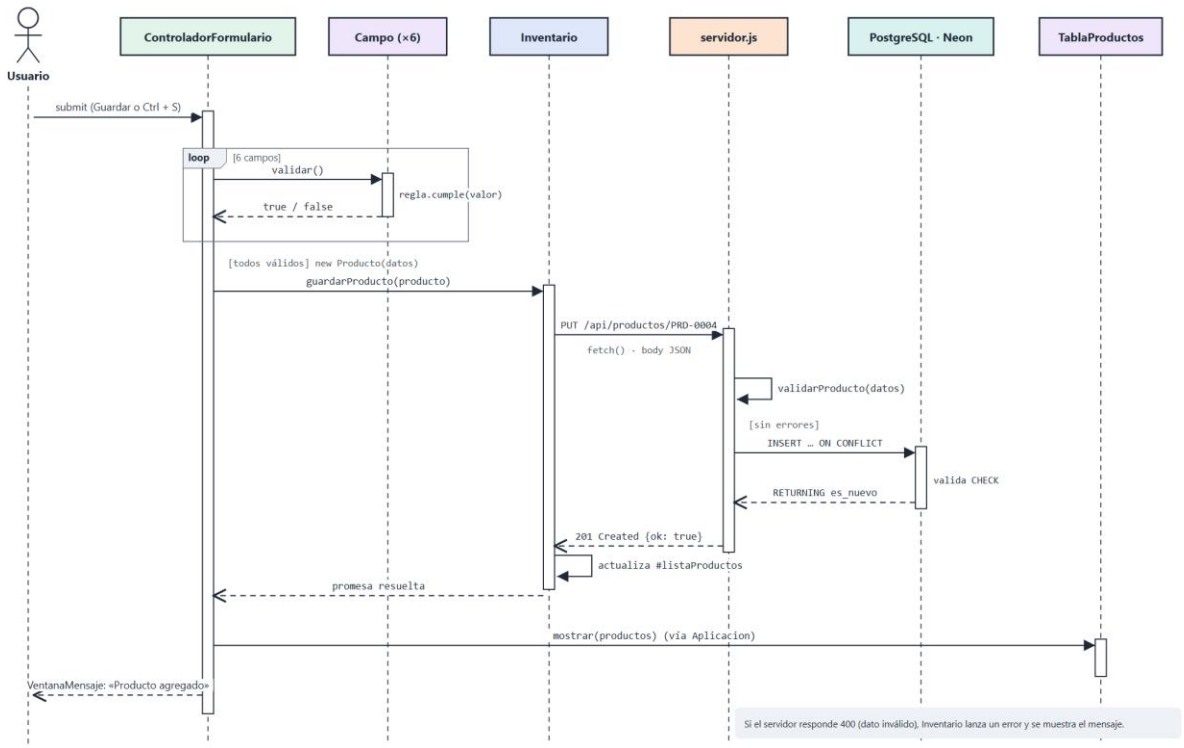
En el servidor, la función `validarProducto()` revisa otra vez cada dato y responde con el código 400 si algo es incorrecto. En la base de datos, las restricciones `CHECK` son la última barrera. Esta defensa en profundidad sigue la recomendación de no confiar solo en la validación del cliente (OWASP Foundation, s.f.).

Diagrama de Secuencia: Guardar un Producto

La Figura 18 muestra, con un diagrama de secuencia UML, cómo colaboran los objetos y el servidor cuando el usuario guarda un producto nuevo. Las flechas continuas son mensajes (llamadas a métodos o peticiones) y las punteadas, respuestas. Primero el controlador valida los seis campos; si todos son correctos, `Inventario` envía la petición `PUT` con `fetch`, el servidor vuelve a validar y PostgreSQL ejecuta el `INSERT` comprobando sus restricciones. Recién cuando la respuesta 201 llega al navegador se actualiza la tabla y se muestra el mensaje «Producto agregado».

Figura 18

Diagrama de secuencia UML al guardar un producto



Nota. Elaboración propia. El código PRD-0004 es un ejemplo; el recuadro «loop» indica que la validación se repite para los seis campos.

Despliegue en Internet

El código se publicó en el repositorio de GitHub `franconex/Actividad-2-DW2` y se desplegó como servicio web gratuito en Render (Render, s.f.). Render descarga el repositorio, instala las dependencias y ejecuta el servidor, mientras que la base de datos permanece en Neon. La Tabla 13 muestra la configuración usada.

Tabla 13

Configuración del servicio web en Render

Campo	Valor	Motivo
Repository	`franconex/Actividad-2-DW2 (main)`	Render se actualiza con cada git push a main.
Root Directory	`proyecto`	El código del sistema está en esa carpeta.
Build Command	`npm install`	Instala express y pg.
Start Command	`node servidor.js`	npm start lee el archivo .env, que no está en GitHub.

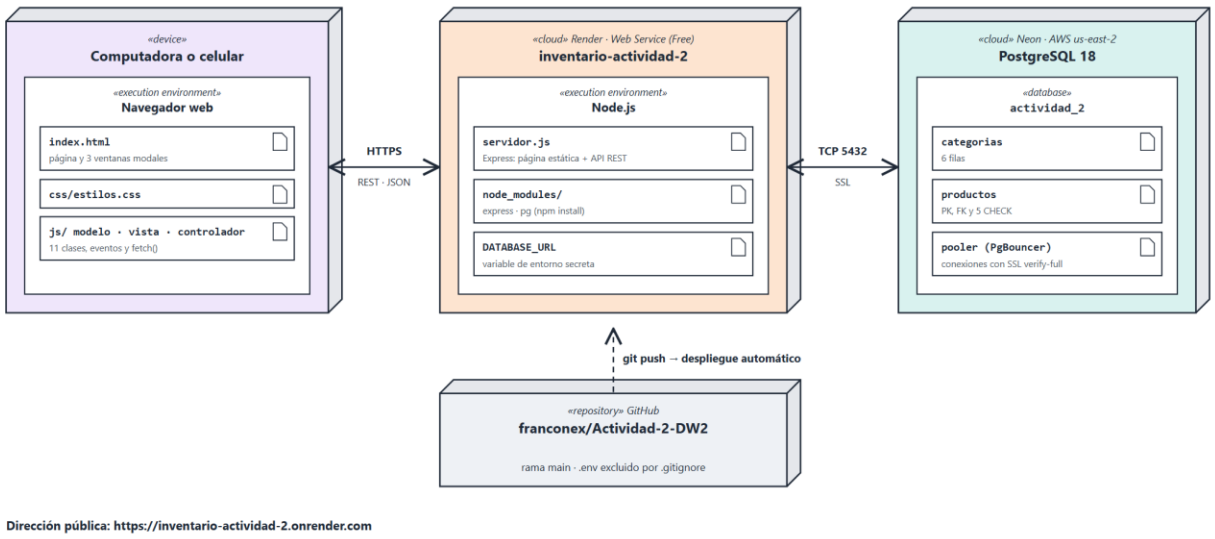
Campo	Valor	Motivo
Environment	`DATABASE_URL`	Cadena de conexión de Neon, guardada como secreto.
Instance Type	Free	Plan gratuito.

Nota. Elaboración propia. Dirección pública del sistema: https://inventario-actividad-2.onrender.com

La Figura 19 presenta el diagrama de despliegue UML, es decir, los nodos físicos o en la nube donde se ejecuta cada parte del sistema y cómo se comunican. El navegador del usuario descarga la página desde Render por HTTPS; el servidor de Render se conecta a Neon por el puerto 5432 con SSL; y GitHub entrega el código a Render en cada `git push`.

Figura 19

Diagrama de despliegue UML del sistema



Nota. Elaboración propia. Cada caja tridimensional es un nodo; las cajas con la esquina doblada son artefactos (archivos o configuraciones).

El archivo `.env`, que contiene la contraseña de la base de datos, está incluido en `.gitignore`, por lo que nunca se sube a GitHub; en Render la misma información se guarda como variable de entorno. Cada vez que se envía un cambio a la rama `main` con `git push`, Render vuelve a desplegar el sistema de forma automática. Una vez publicado, se comprobó que la dirección https://inventario-actividad-2.onrender.com muestra la página y que la ruta

`/api/productos` devuelve los productos almacenados en Neon. En el plan gratuito, el servicio se suspende tras 15 minutos sin visitas, por lo que la primera carga posterior puede tardar alrededor de un minuto.

Resultados y Pruebas

Funcionamiento del Sistema

A continuación se presentan las capturas del sistema en funcionamiento, tomadas en el navegador Google Chrome.

Figura 20

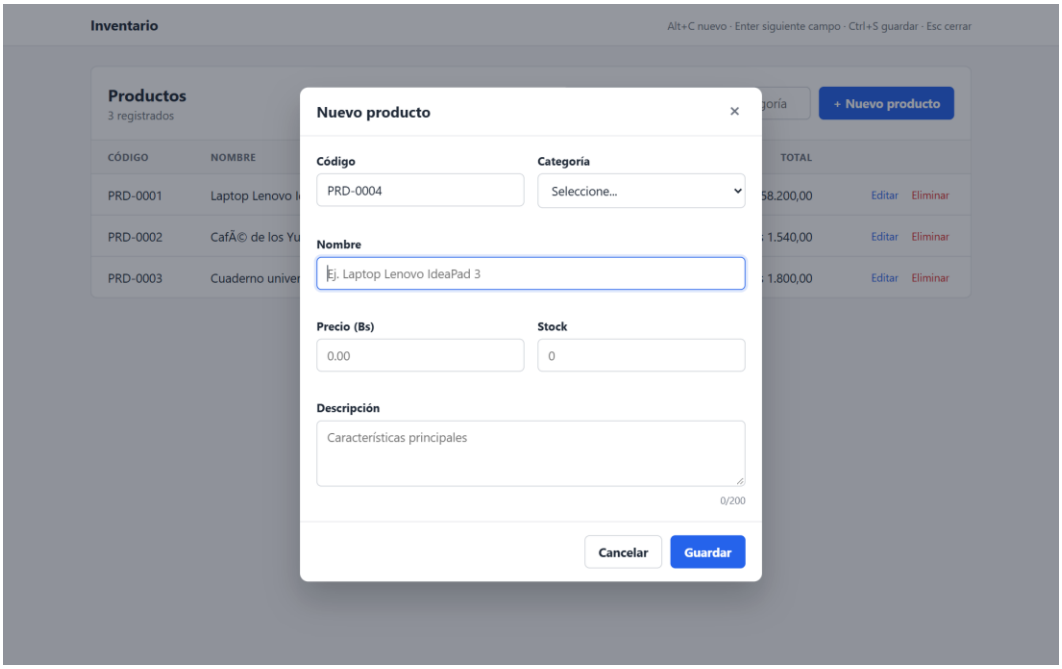
Pantalla principal con el listado de productos



Nota. La barra superior recuerda los atajos de teclado. Cada fila tiene las acciones Editar y Eliminar.

Figura 21

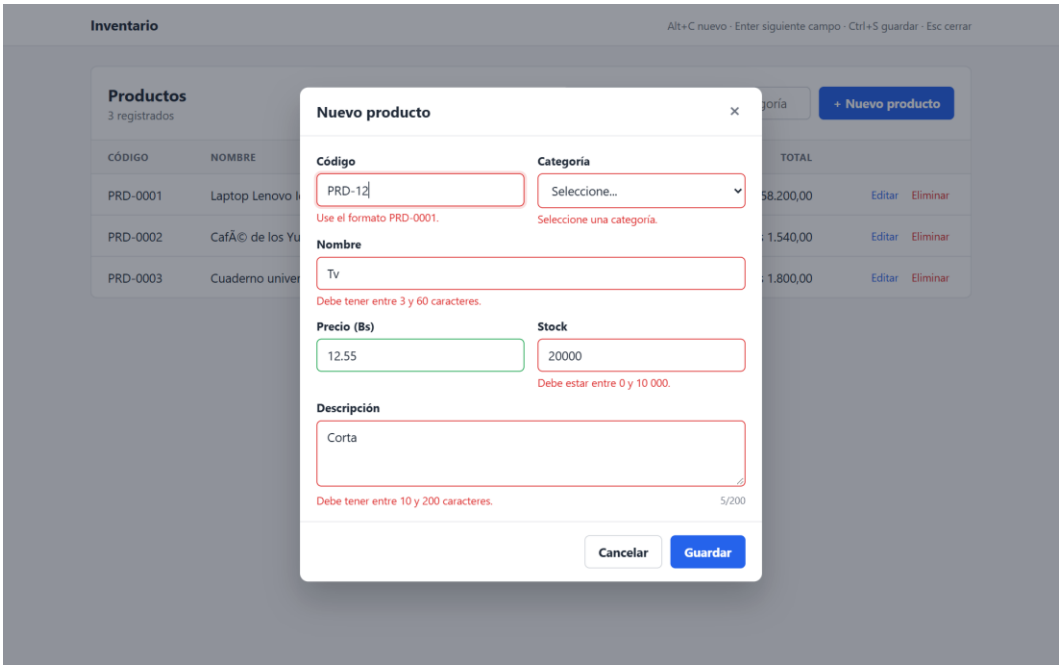
Ventana modal para registrar un producto nuevo



Nota. El código PRD-0004 se generó automáticamente y el cursor se ubica en el campo Nombre.

Figura 22

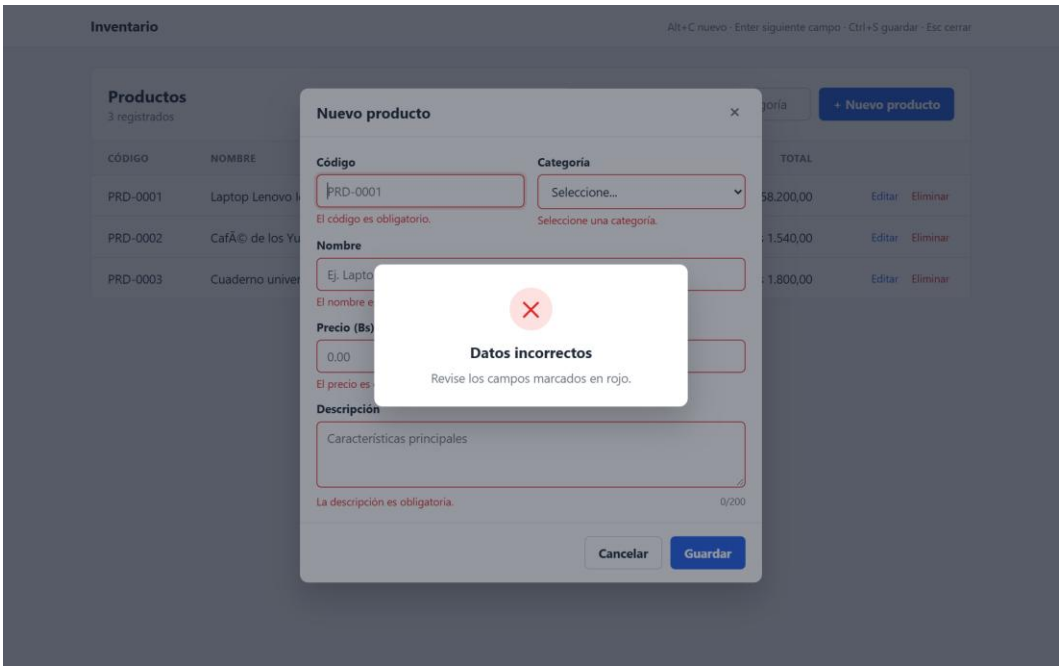
Validación interactiva con datos incorrectos



Nota. Cada campo muestra su borde rojo y el mensaje del primer error. El precio 12.55 es válido (borde verde); al escribirlo se intentaron ingresar letras y un segundo punto, que fueron bloqueados por `beforeinput`.

Figura 23

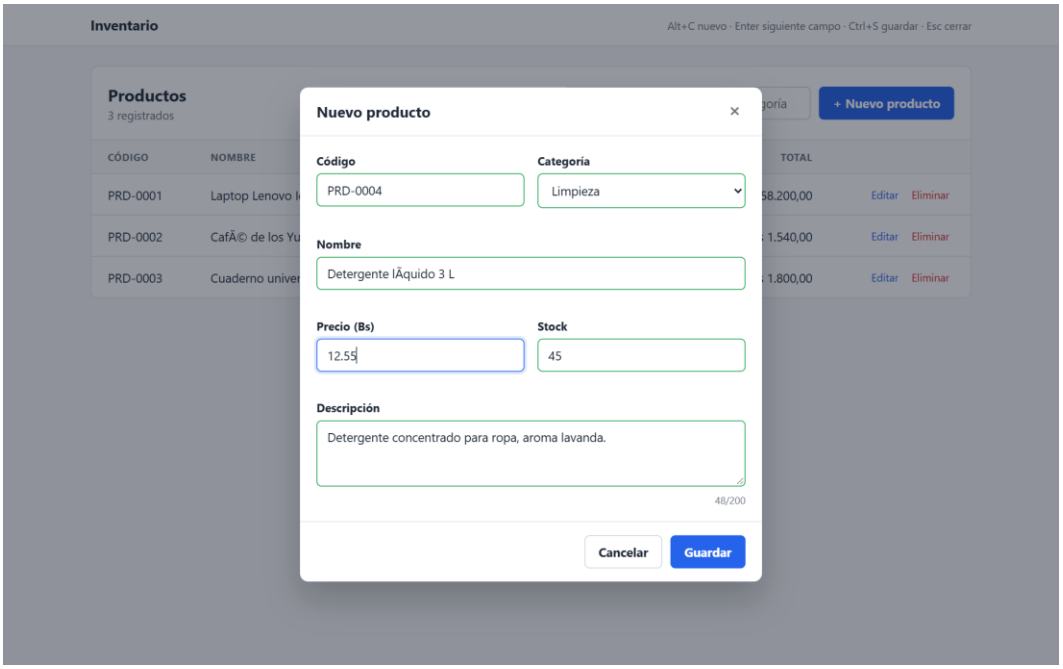
Mensaje temporal al intentar guardar con errores



Nota. La ventana “Datos incorrectos” se cierra sola a los 2 segundos y el formulario permanece abierto.

Figura 24

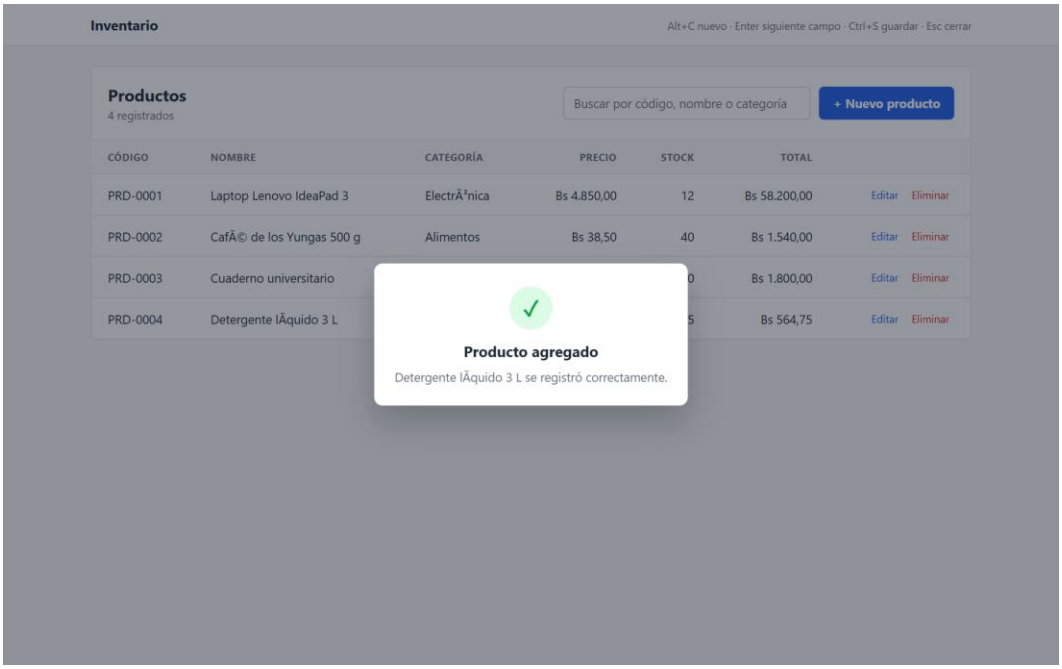
Formulario con todos los datos correctos



Nota. Todos los campos tienen borde verde y el contador indica 48/200 caracteres.

Figura 25

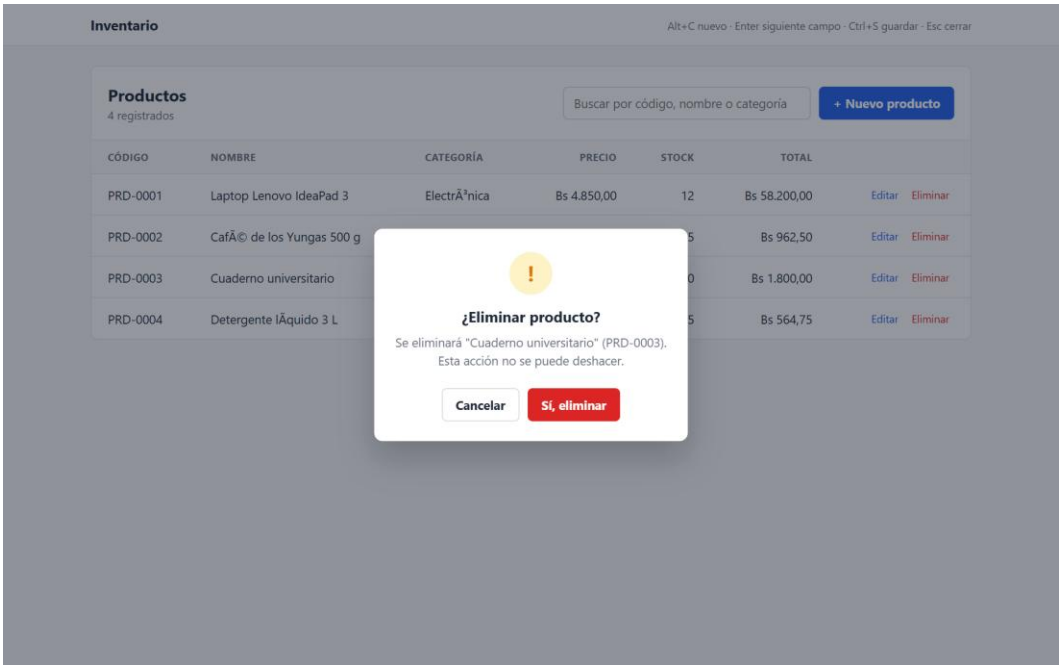
Mensaje temporal de producto agregado



Nota. Se guardó con Ctrl + S; la tabla ya muestra el nuevo producto PRD-0004.

Figura 26

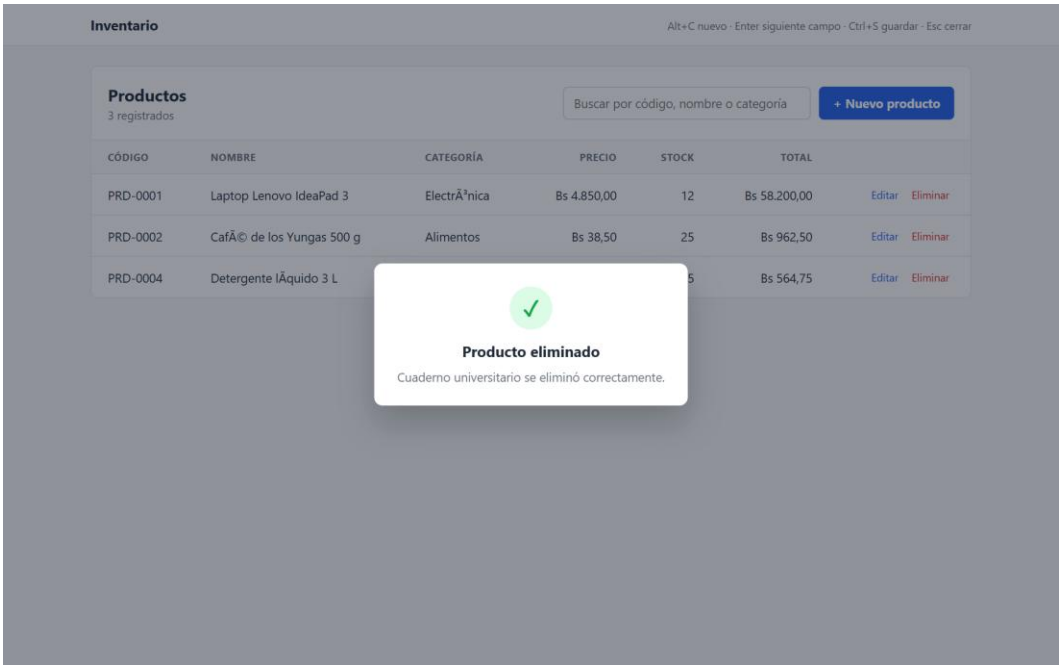
Ventana de confirmación antes de eliminar



Nota. Si el usuario presiona Cancelar, Esc o hace clic afuera, el producto no se elimina.

Figura 27

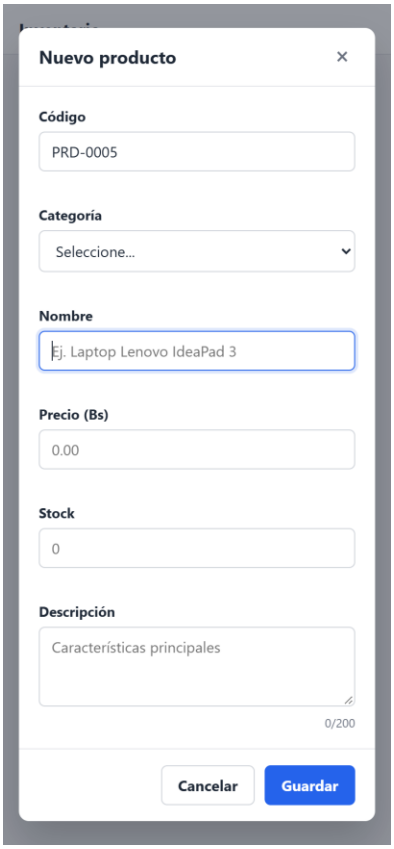
Mensaje temporal de producto eliminado



Nota. La tabla se actualizó y el contador muestra 3 productos registrados.

Figura 28

Vista del formulario en un teléfono celular



Nota. En pantallas pequeñas el formulario pasa a una sola columna.

Pruebas Funcionales

Las pruebas se ejecutaron de forma automatizada en Google Chrome, simulando pulsaciones reales de teclado y clics de mouse. Los 35 casos de la Tabla 14 resultaron aprobados y no se registraron errores de JavaScript en la consola. Durante una versión anterior, estas pruebas detectaron un error real: al editar un producto, su propio código aparecía como "repetido". Se corrigió asignando `#codigoEnEdicion` antes de validar, lo que demuestra la utilidad de probar el sistema.

Tabla 14

Casos de prueba ejecutados

N.º	Caso de prueba	Evento	Resultado esperado	Estado
1	Cargar la página con 3 productos guardados	—	Se muestran 3 filas	Aprobada
2	Presionar Alt + C	keydown	Se abre la ventana del formulario	Aprobada
3	Abrir “Nuevo producto”	click	Código PRD-0004 generado	Aprobada
4	Abrir el formulario	—	Cursor en el campo Nombre	Aprobada
5	Presionar Esc	keydown	La ventana se cierra	Aprobada
6	Clic en “+ Nuevo producto”	click	La ventana se abre	Aprobada
7	Clic en el fondo oscuro	mousedown	La ventana se cierra	Aprobada
8	Guardar con todo vacío	submit	6 campos en rojo	Aprobada
9	Guardar con errores	submit	Mensaje “Datos incorrectos”	Aprobada
10	Esperar 2 segundos	—	El mensaje se cierra solo	Aprobada
11	Mensaje cerrado	—	El formulario sigue abierto	Aprobada
12	Escribir “prd-12” en Código	input	Se convierte a “PRD-12”	Aprobada
13	Escribir “1a2.b5.5x” en Precio	beforeinput	Queda “12.55”	Aprobada
14	Escribir “20000x” en Stock	beforeinput	Queda “20000”	Aprobada
15	Código “PRD-12”	focusout	“Use el formato PRD-0001.”	Aprobada
16	Nombre “Tv”	focusout	“Debe tener entre 3 y 60 caracteres.”	Aprobada
17	Stock 20000	focusout	“Debe estar entre 0 y 10 000.”	Aprobada
18	Código “PRD-0001” (ya existe)	focusout	“Ya existe un producto con este código.”	Aprobada
19	Enter en Código	keydown	El cursor pasa a Categoría	Aprobada
20	Corregir el nombre	input	Borde verde en vivo	Aprobada
21	Escribir la descripción	input	Contador 48/200	Aprobada
22	Presionar Ctrl + S con datos correctos	keydown	Se guarda y se cierra; 4 filas	Aprobada
23	Después de guardar	—	Mensaje “Producto agregado”	Aprobada
24	Doble clic en una fila	dblclick	Ventana “Editar producto”	Aprobada
25	Editar producto existente	—	Código bloqueado, sin error de duplicado	Aprobada
26	Cambiar stock y guardar	click	Mensaje “Producto actualizado”	Aprobada
27	Revisar la tabla	—	Stock actualizado a 25	Aprobada
28	Clic en Eliminar	click	Ventana “¿Eliminar producto?”	Aprobada
29	Clic en Cancelar	click	No se elimina (4 filas)	Aprobada
30	Presionar Esc en la confirmación	keydown	No se elimina (4 filas)	Aprobada
31	Clic en “Sí, eliminar”	click	Se elimina (3 filas)	Aprobada
32	Después de eliminar	—	Mensaje “Producto eliminado”	Aprobada
33	Escribir “café” en el buscador	input	Queda 1 fila	Aprobada
34	Recargar la página	—	Se conservan 3 productos	Aprobada

N.º	Caso de prueba	Evento	Resultado esperado	Estado
35	Abrir en pantalla de 390 px	—	Sin desplazamiento horizontal	Aprobada

Nota. Elaboración propia. Pruebas automatizadas con Chrome sobre el archivo `index.html`.

Uso de Inteligencia Artificial

La consigna permite el uso de inteligencia artificial siempre que se describan los prompts empleados. Se utilizó el asistente Claude (Anthropic, 2026) como apoyo para investigar, generar y revisar el código. Un **prompt** es la instrucción que se le da a la IA; redactarlo bien —con contexto, restricciones y un formato esperado— mejora notablemente la respuesta (White et al., 2023). La Tabla 15 presenta los prompts utilizados, con la ortografía corregida, y la técnica que aplica cada uno.

Tabla 15

Prompts redactados y técnica aplicada

N.º	Prompt (resumen fiel)	Técnica de prompting	Resultado obtenido
1	“Necesito que me ayudes con esto a sacar 100”, junto con la consigna completa y la rúbrica.	Contexto completo: se entregan los criterios de evaluación para que la IA conozca el objetivo.	Primera versión del sistema con POO, eventos y pruebas.
2	“Mejora el formulario: más profesional, sencillo y ordenado; sin monitor de eventos ni barra de progreso; tipo modal; código sencillo y bien comentado con comentarios claros y cortos.”	Restricciones explícitas (qué quitar y qué conservar) y estilo de salida.	Diseño limpio con el formulario en una ventana modal.
3	“Código y variables en español con nombres referentes; agrega ventanas temporales: ¿seguro de borrar?, producto agregado correctamente; ordénalo en la menor cantidad de archivos con una estructura puntual.”	Refinamiento iterativo: se corrige el resultado anterior con requisitos nuevos.	Estructura MVC en 3 archivos y ventanas de confirmación y mensaje.
4	“Cambia el CSS, hazlo hacia abajo (una propiedad por línea).”	Formato de salida específico.	CSS en 11 secciones, una propiedad por línea.
5	“¿Esto es lo más sencillo que puedes hacer o con esto cumplimos lo que pide para sacar 100?”	Verificación: se pide a la IA evaluar su propio resultado frente a la rúbrica.	Análisis de cumplimiento y de lo que faltaba (el informe).
6	“Haz el documento explicando cada clase y función, con imágenes y código, máximo 40 hojas, con carátula; recuerda que debo defender el código.”	Rol del destinatario y límites (extensión, propósito de defensa).	El presente informe en APA 7 con guía para la defensa.
7	“En GitHub súbelo solo para que esté desplegado… ¿qué sugieres tú?” y “Guíame para conectar Render paso a paso.”	Petición de recomendación y descomposición en pasos: se pide una guía secuencial y justificada.	Despliegue en Render con la base de datos en Neon.

N.º	Prompt (resumen fiel)	Técnica de prompting	Resultado obtenido
8	“Actualízame la diapositiva y el documento para ya contemplar la base de datos y el despliegue.”	Actualización con contexto: se pide adaptar entregables existentes a los cambios del proyecto.	Informe y presentación actualizados.
9	“No veo esos cambios… agrega UML; no hablas nada de Neon ni de la base de datos.”	Retroalimentación correctiva: se indica qué falta para que la IA amplíe el resultado.	Diagramas UML de clases, secuencia y despliegue, modelo entidad–relación y sección sobre Neon.

Nota. Elaboración propia. Los prompts se transcriben de la sesión de trabajo con la IA.

El uso de la IA se realizó con responsabilidad: cada versión del código se ejecutó y se probó en el navegador; se revisó que el programa cumpliera la consigna; y se estudió cada clase para poder explicarla. La IA fue una herramienta de apoyo y no un reemplazo del aprendizaje, en línea con el eje transversal de Responsabilidad Social y Ética.

Guía para la Defensa del Proyecto

Esta sección reúne las preguntas que con mayor probabilidad se plantearán durante la defensa, con una respuesta breve basada en el código.

1. ¿Qué es una clase y qué es un objeto en tu proyecto?

Una clase es la plantilla; un objeto es un elemento creado con ella. `Producto` es la clase y cada producto registrado, como `new Producto({ codigo: 'PRD-0001', ... })`, es un objeto.

2. ¿Dónde aplicas encapsulamiento y para qué sirve el símbolo #?

En `Producto`, `Inventario`, `Campo` y `Ventana`. El `#` hace que el atributo sea privado: si desde fuera se escribe `producto.#precio = -5`, JavaScript marca un error. Así nadie puede dejar un producto con datos inválidos.

3. ¿Por qué Regla es una clase abstracta?

Porque representa la idea general de “regla”, no una regla concreta. Si se intenta `new Regla()`, el constructor detecta con `new.target` que no es una subclase y lanza un error. Solo se crean sus hijas: `ReglaObligatorio`, `ReglaLongitud`, etc.

4. ¿Dónde está la herencia y dónde el polimorfismo?

Herencia: las 5 reglas usan `extends Regla` y las ventanas de confirmación y mensaje usan `extends Ventana`. Polimorfismo: `Campo.validar()` llama `regla.cumple(valor)` y cada regla responde a su manera; además, `VentanaConfirmacion` sobrescribe `cerrar()`.

5. ¿Cuál es la diferencia entre keydown, beforeinput e input?

`keydown` ocurre al presionar cualquier tecla y sirve para atajos. `beforeinput` ocurre antes de que el texto cambie, por eso permite bloquear una letra con `preventDefault()`. `input` ocurre después del cambio y sirve para validar en vivo. No se usó `keypress` porque está obsoleto.

6. ¿Qué es la delegación de eventos y dónde la usas?

Es registrar un solo controlador en el elemento padre en lugar de uno por cada hijo. En `TablaProductos` hay un único `click` en el cuerpo de la tabla que atiende todos los botones Editar y Eliminar, incluso los de filas que se agregan después.

7. ¿Para qué sirve preventDefault()?

Para cancelar la acción por defecto del navegador: en `submit` evita que la página se recargue; en Ctrl + S evita que se abra “Guardar página”; en `beforeinput` evita que se escriba el carácter.

8. ¿Cómo funciona la ventana “¿Eliminar producto?”?

`preguntar()` devuelve una promesa. En `Aplicacion`, `await` espera esa respuesta: si es `true` se elimina; si es `false`, no. Así el código se lee en orden, como si esperara al usuario.

9. ¿Dónde se guardan los datos?

En una base de datos PostgreSQL alojada en Neon, en las tablas `productos` y `categorias`. La página no accede directamente: la clase `Inventario` usa `fetch` para llamar a la API de `servidor.js`, y el servidor ejecuta las consultas SQL.

10. ¿Es suficiente validar en el navegador?

No. Mejora la experiencia, pero el usuario puede desactivar JavaScript (OWASP Foundation, s.f.). Por eso las mismas reglas se repiten en el servidor, con `validarProducto()`, y en la base de datos, con restricciones `CHECK`.

11. ¿Cómo agregarías un campo o una regla nueva?

Para una regla nueva se crea otra clase que herede de `Regla` e implemente `cumple()`. Para un campo nuevo se agrega el control en el HTML y una línea `new Campo(...)` en `#crearCampos()`. No hay que modificar las demás clases.

12. ¿Por qué usaste el patrón MVC?

Para separar responsabilidades: el modelo maneja los datos, la vista lo que se ve y el controlador los eventos. Esto hace que el código sea más ordenado y que un cambio en una parte no rompa las otras.

13. ¿Cómo se publicó el sistema en internet?

El código está en GitHub y Render lo despliega como servicio web: ejecuta `npm install` y `node servidor.js`. La base de datos está en Neon y se conecta con la variable de entorno `DATABASE_URL`. Cada `git push` a `main` actualiza la página.

14. ¿Por qué la contraseña de la base de datos no está en GitHub?

Porque el archivo `.env` está en `.gitignore`. Si se subiera, cualquiera podría leer o borrar los datos. En Render la contraseña se guarda como variable de entorno secreta.

15. ¿Qué es Neon y por qué se usó?

Es un servicio de PostgreSQL en la nube de tipo *serverless*: el cómputo se suspende cuando no hay consultas y se reactiva solo. Se usó porque es gratuito, no requiere instalar PostgreSQL y entrega una cadena de conexión cifrada con SSL.

16. ¿Qué muestra el diagrama de secuencia?

El orden de los mensajes al guardar: el controlador valida los campos, `Inventario` envía `PUT` con `fetch`, el servidor valida otra vez, PostgreSQL inserta y revisa los `CHECK`, y al volver la respuesta 201 se actualiza la tabla.

Ejes Transversales y Objetivos de Desarrollo Sostenible

Tabla 16

Vinculación con los ejes transversales y los ODS

Eje u ODS	Vinculación con el proyecto
Tecnologías emergentes e IA	Uso documentado de IA generativa y técnicas de redacción de prompts.
Emprendedurismo e innovación	El sistema puede servir de base para el control de inventario de pequeños negocios sin costo de licencias.
Investigación en el aula	Investigación de características del lenguaje, eventos y buenas prácticas, contrastada con fuentes oficiales.
Responsabilidad social y ética	Uso honesto de la IA; protección contra la inyección de código; formulario usable solo con teclado.
ODS 4: Educación de calidad	Desarrollo de competencias digitales aplicadas a un problema real (Naciones Unidas, 2015).
ODS 8: Trabajo decente y crecimiento económico	Herramienta que apoya la gestión de microempresas.
ODS 9: Industria, innovación e infraestructura	Software web ligero que funciona en cualquier navegador, sin infraestructura costosa.
ODS 10: Reducción de las desigualdades	Diseño adaptable a celulares y uso completo con teclado, lo que mejora el acceso.

Nota. Elaboración propia.

Conclusiones

Se cumplió el propósito del taller: se desarrollaron módulos lógicos orientados a objetos y controladores de eventos de teclado y mouse que validan de forma interactiva el formulario de productos. Las once clases aplican los cuatro pilares de la POO y se organizan con el patrón MVC en solo tres archivos de JavaScript.

La investigación permitió elegir los eventos adecuados para cada necesidad: `beforeinput` para impedir caracteres inválidos antes de que aparezcan, `input` y `focusout` para

validar en el momento justo, `keydown` para los atajos y la delegación de eventos para la tabla. Asimismo, características modernas como los campos privados, las clases abstractas y las promesas hicieron el código más seguro y legible.

Las 35 pruebas aprobadas confirman el correcto funcionamiento de la interfaz, y el proceso de pruebas permitió detectar y corregir un error real.

Finalmente, el sistema se conectó a una base de datos PostgreSQL alojada en Neon mediante una API REST y se publicó en internet con Render. Los diagramas UML de clases, de secuencia y de despliegue, junto con el modelo entidad–relación, documentan cómo colaboran las tres capas. Con ello, los datos son compartidos y permanentes, y las reglas de validación se aplican en tres niveles: navegador, servidor y base de datos.

Referencias

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Ecma International. (2024). *ECMAScript® 2024 language specification* (15.ª ed.). https://262.ecma-international.org/15.0/

Flanagan, D. (2020). *JavaScript: The definitive guide* (7.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Haverbeke, M. (2024). *Eloquent JavaScript: A modern introduction to programming* (4.ª ed.). No Starch Press.

Mozilla. (s.f.-a). *Clases*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes

Mozilla. (s.f.-b). *Propiedades privadas*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/Private_properties

Mozilla. (s.f.-c). *EventTarget.addEventListener()*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener

Mozilla. (s.f.-d). *Introducción a los eventos*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Learn/JavaScript/Building_blocks/Events

Mozilla. (s.f.-e). *Element: beforeinput event*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/Element/beforeinput_event

Mozilla. (s.f.-f). *Validación de formularios del lado del cliente*. MDN Web Docs. Recuperado el 5 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Learn/Forms/Form_validation

Mozilla. (s.f.-g). *Uso de Fetch*. MDN Web Docs. Recuperado el 6 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Naciones Unidas. (2015). *Transformar nuestro mundo: La Agenda 2030 para el Desarrollo Sostenible* (A/RES/70/1). https://www.un.org/sustainabledevelopment/es/

Neon. (s.f.). *Neon documentation*. Recuperado el 6 de noviembre de 2026, de https://neon.com/docs

OpenJS Foundation. (s.f.). *Express: Framework web rápido, minimalista y flexible para Node.js*. Recuperado el 6 de noviembre de 2026, de https://expressjs.com/es/

OWASP Foundation. (s.f.). *Input validation cheat sheet*. Recuperado el 5 de noviembre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

PostgreSQL Global Development Group. (s.f.). *Constraints*. PostgreSQL documentation. Recuperado el 6 de noviembre de 2026, de https://www.postgresql.org/docs/current/ddl-constraints.html

Render. (s.f.). *Deploy a Node Express app on Render*. Recuperado el 6 de noviembre de 2026, de https://render.com/docs/deploy-node-express-app

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). *A prompt pattern catalog to enhance prompt engineering with ChatGPT* (arXiv:2302.11382). arXiv. https://doi.org/10.48550/arXiv.2302.11382

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/